# SciTeam：让一群 AI 像一个靠制度运转的研究团队那样工作

我们在问：只改「团队规章」，不改程序代码， AI 团队的产出会不会变好?

研究进展汇报 · 2026-10-19 · 最新实验 X-evid 为中期数据（已完成 252/432 次运行）

# 汇报提纲

- 一、我们想回答什么：目标与最初的设想
- 二、怎么回答：一串环环相扣的问题 + 保证结果可信的实验规矩
- 三、 SciTeam 系统长什么样
- 四、各实验的结果（按时间顺序，每个实验回答上一个留下的问题）
- 五、踩过的坑
- 六、有价值、有趣的发现
- 七、下一步

# 先认识几个关键词

后面每页都会用到；遇到不懂的可以回到这页

| 关键词 | 大白话解释 |
| --- | --- |
| 智能体（ agent ） | 大模型 + 工具（读写文件、跑代码、发消息），能自己连续干活 |
| 制度 / 规章 | 写在配置文件里的团队规则，如「批准人不能是作者」；程序只负责照章执行 |
| 关闭 / 虚假关闭 | 团队宣布「做完了」 / 宣布做完了，但官方答案判定**其实没全对** |
| 精确率 | 团队说「做完了」的任务中，**真的全对**的比例 —— 团队的话有多可信 |
| 独立证据 | 检查者手里有作者**看不到**的信息（例如一份保留测试题） |
| 臂 | 实验中的一种条件；各臂只差一个因素，用来看这个因素的作用 |
| 断路器 | 花费 / 时间上限的保险丝；被触发 = 系统有缺陷，要排查 |
| 重放审计 | 事后用同样规则把全部动作重放一遍，结果必须完全一致 |

# 一、我们想做成的两件事

- **工程目标**：给一个「创始人」 AI 一句使命，它能自己写章程、招人、分工，完成真实的研究与工程任务
- **科学目标**：程序冻结不动，**只换规章文件**，观察团队表现怎么变 —— 结果不同，就一定是规章造成的
- 我们的立场：团队怎么运作由**制度**决定，而不是由程序员把流程写死
  - 程序里没有「部门」「角色」这类概念，全部写在配置里
  - 「这份工作算不算通过」不让大模型拍板，由规则 + 机械测试结果决定
- 标志性目标：把 Google 已发表的多智能体科研组织 **Cogentic** ，不写专门代码、只用规章「装」进来，并复现其效果

# 一、立项时的设想，现在怎么样了

有的被证实，有的被推翻；被推翻的那些反而让研究方向更清楚

| 立项时的设想 | 现在的结论 |
| --- | --- |
| 加一个审核员 + 返工流程，质量就会提高 | **不成立**：审核员和作者看同样的东西，等于自己审自己 |
| 审核员手里有独立的检查材料，就能挡住错误 | **成立**：「做完了」的任务中全对比例 8% → 93% |
| 返工卡住时，可以由程序自动判断并叫停 | **不成立**：程序叫停更糟；把放弃权交给审核员才有效 |
| 两个 AI 互不知情各做一份、比对结果，能自产证据 | **进行中**：错误的「完成」减半，但可信度没提高（后详） |
| 换不同厂商的模型，独立性更强 | 方向一致但很弱，还不能下结论 |
| 不同协作模式（层级、扁平…）效果不同 | 原任务太简单都满分，推迟到更难的任务 |

# 二、研究设计：一串环环相扣的问题

每个实验都回答上一个实验留下的问题

| 编号 | 问题 | 实验 | 状态 |
| --- | --- | --- | --- |
| Q1 | 给团队加审核手续，能不能提高质量? | X-inst | 完成：不能 |
| Q1a | 审核员有独立证据时，能挡住多少错误？代价? | X-inst-confirm 等 | 完成：能挡住大部分 |
| E3 | 两份独立解答比对结果，理论上能发现错误吗? | 离线预研 | 完成：能发现一部分 |
| Q1b | 没有标准答案时，团队能否**自己**制造独立证据? | X-evid | **进行中 252/432** |
| Q5 | 能否只用规章复现 Google 的 Cogentic 组织? | X-cog | 已能装配运行 |
| Q2–Q4 | 协作模式比较 / 组织自我进化 / 真实科研任务 | X-mode / X-evo / X-sci | 排队中 |

# 二、为了让结果可信，我们给自己定的规矩

- **只改规章不改程序**：各组实验的「程序指纹」必须完全相同
- **先登记后运行**：假设、指标、多大差别才算数、什么结果算失败 —— 跑之前写死，跑完不许改
- **外部判分**：用公开基准的官方判分程序，**不用 AI 给 AI 打分**
- **重放审计**：每次运行事后重放一遍，结果不一致的不计入
- **撞保险丝 = 系统缺陷**：必须查清原因，不能算作「模型不行」
- **偏差登记**：中途发现的问题全部编号登记（目前 D1–D19），受影响的运行作废但保留
- **七问复盘**：每个实验结束回答七个问题，其中最关键的是「这个实验**实际**回答了什么」

# 三、 SciTeam 系统：五层，自下而上搭

每一层都先用真实模型通过验收，才开始写上一层

| 层 | 管什么（大白话） | 验收 |
| --- | --- | --- |
| 1 单个 AI | 一个 AI 怎么循环干活、上下文太长怎么压缩、预算控制、崩溃恢复 | H1–H8 通过 |
| 2 通信 | 私聊、群聊、 @ 人、任务的领取 / 提交 / 批准；有消息就叫醒对应的 AI | C1–C6 通过 |
| 3 制度 | 所有动作经同一个「闸门」按规章审批；什么算「通过」由规则决定 | I1–I6 通过 |
| 4 组织 | 创始人、章程、招聘、目标；目标达成自动停止 | O1–O5 通过 |
| 5 实验 | 定义实验条件、批量运行、外部判分、统计分析 | 进行中 |

上层可以用下层，下层不能依赖上层 —— 保证每一层能单独测试、单独替换。

# 三、系统怎么运转：所有动作都要过同一道闸门

- ① 某个 AI 收到 @ 、私信或新任务 → 调度器把它「叫醒」
- ② 它自己思考、读文件、写代码、跑测试
- ③ 凡是影响别人的动作（发消息、提交、批准、改规章、招人）→ 打包成「申请」交给**闸门**
- ④ 闸门按当前规章：**放行 / 拒绝 / 等人批准**；被拒也记录在案，并告诉 AI 原因
- ⑤ 满足规章条件（如「测试通过 + 非作者批准」）→ 系统记一条「验收通过」的**正式记录**
  - 这条记录由规则推出，**不由任何 AI 说了算**
- ⑥ 新记录叫醒其他相关 AI，循环往复，直到目标达成
- 所有动作按顺序记录 → 任何时刻都能重建 → 可以**重放审计**

# 三、规章能写什么：五类规则 + 一条原则

| 规则类型 | 作用（类比公司） | 例子 |
| --- | --- | --- |
| 检查器 | 在干净环境里跑命令，看成败；不用 AI | 跑测试；比较两份代码的输出 |
| 前置条件 | 做某件事之前必须满足的条件 | 提交前测试必须通过；批准人 ≠ 作者 |
| 审批 | 某些动作需特定的人签字 | 正式发布须人类签字 |
| 认定规则 | 什么情况「算作」什么 | 完成 + 测试过 + 非作者批准 ⇒ 验收通过 |
| 账本 | 记功记过（只记账，不自动惩罚） | 交付 +10 ，被退回 − 3 |

**原则：程序只核对事实，判断交给角色。** 「还值不值得返工」「该不该放弃」由审核员等角色决定；规章只规定谁有权判断、要写明什么理由、判断后有什么后果。

# 三、用一个例子看懂我们的实验

以最新的 X-evid 实验为例：同一道科学计算题，三种团队规章

| | 对照组 | 同模型双盲 | 跨模型双盲 |
|---|---|---|---|
| 两名实现者 | flash + flash ，互相看不见、不能交流 | flash + flash ，互相看不见、不能交流 | flash + mimo ，互相看不见、不能交流 |
| 怎么算「完成」 | 两人都交卷就算完成 | 审核员比对两人输出，**一致才算完成** | 同左 |
| 不一致怎么办 | — | 只告诉「第几步不一致」，各自再查一轮；仍不一致 → **审核员放弃** | 同左 |
| 要回答什么 | 排除「只是多花一份算力」 | 同模型的独立重复能否发现错误 | 换个模型是否更独立 |

无论团队说什么，最后都用**官方标准答案**给 1 号的答案打分 → 算出团队说「完成」时到底对不对。 24 题 × 3 种规章 × 6 次 = 432 次运行。

# 四、第一层验收：我们的单个 AI 和业界最好水平持平

SWE-bench Verified： 500 个真实 GitHub 项目的 bug，修好且通过官方测试才算解决

| 系统（同一模型） | 解决题数 | 费用 | 说明 |
| --- | --- | --- | --- |
| SciTeam （我们） | 414 / 500 （82.8%） | **$28.68** | 费用最低 |
| pi （开源强基线） | 415 / 500 （83.0%） | ⩽ $37.83 | 费用为上限估计 |
| Hermes （开源强基线） | 414 / 500 （82.8%） | ⩽ $63.95 | 费用为上限估计 |

- 三家差距 ⩽ 1 题，属于随机波动；我们的优势在费用
- 命令行任务基准 Terminal-Bench 2.1 （重复 5 遍）： **84.3%** ，官方参考 82.7%
- 顺带发现：开发用题库残留 git 历史， AI 能「偷看答案」，成功率被**虚高约 25 个百分点**

# 四、第二到四层验收：通信、制度、组织都在真实运行中跑通

验收任务：一个带 74 条隐藏测试的小型 Python 库

| 层 | 做了什么 | 结果 | 真实运行中发现并修复的问题（举例） |
| --- | --- | --- | --- |
| 2 通信 | 5 种协作方式 × 2 个模型 | flash 全部满分； 6 项通信检查通过 | 过期闹钟导致团队一直不停；两个调度进程同时驱动 |
| 3 制度 | 带审核、带修宪的团队 | 满分；修宪 2 次生效；重放全部一致 | 批准时没附代码版本导致闸门崩溃 |
| 4 组织 | 创始人自建团队 5 次 | 全部自己达成目标并停止；**主动修改规章** | **两份规章修改互相覆盖**（靠重放审计才发现） |

局限：这个任务太简单，所有协作方式都满分 → 比较协作模式需要更难的任务，推迟到后面。

# 四、实验 X-inst：给团队加审核手续，质量没有提高

6 个真实开源库重写任务 × 5 种规章 × 3 次 = 90 次运行

| 与标准版相比，改了什么 | 分数差 | 95% 置信区间（真实差异大概率落在此范围） |
|---|---|---|
| 去掉强制测试 | 0.000 | [−0.027, 0.033] |
| 实现者自己批准自己 | +0.011 | [−0.027, 0.061] |
| 禁止审核员退回返工 | +0.013 | [−0.023, 0.053] |
| 两名审核员都同意才通过 | +0.023 | [−0.010, 0.071] |

- 审核员确实在工作（双审核员那组退回了 24 次），但**最终分数纹丝不动**
- 90 次运行**全部**被批准为「完成」，却**没有一次**真正通过全部隐藏测试 —— 审核成了橡皮图章
- 典型例子：团队自写测试 222 个全过，官方隐藏测试只过 23/38 —— **审核员看的和作者看的是同一批测试**
- 结论：关键不是「有没有审核流程」，而是「审核员有没有**独立的信息来源**」

# 四、实验 X-inst-confirm：给审核员一份作者看不到的测试题

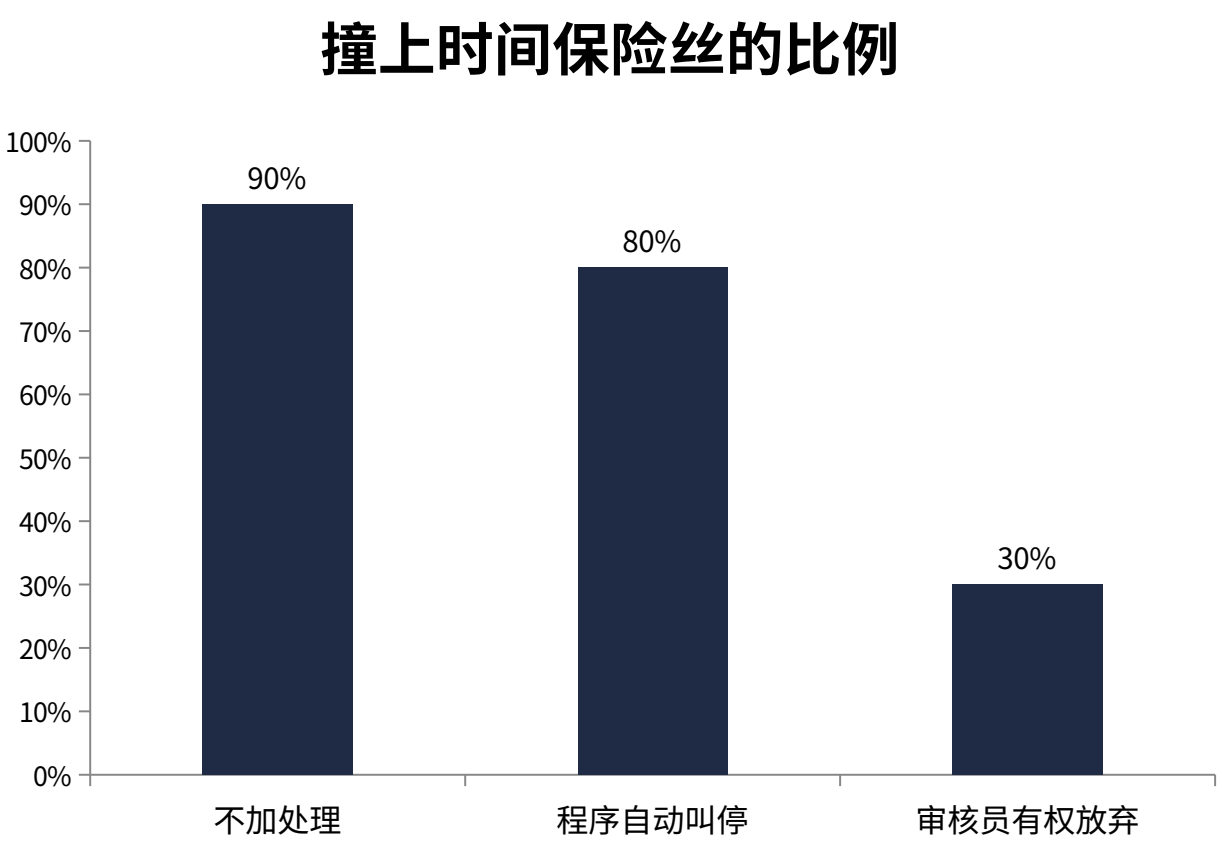
SciCode 科学计算题 24 道 × 3 种规章 × 8 次 = 576 次运行

| 规章 | 平均分 | 说完成但其实错了（次） | 撞时间保险丝（次） |
|---|---|---|---|
| 同证据审核（审核员看同样的测试） | 0.584 | 143 | 1 |
| **独立证据**（审核员有作者看不到的测试） | **0.704** | **27** | **75 （39%）** |
| 安慰剂（只返工，不给对错信息） | 0.548 | 133 | 8 |

- 团队说「完成了」的任务中，真正全对的比例：**8.3% → 92.6%**
- 安慰剂没用：单纯「多返工一次」不起作用，起作用的是**信息**
- 代价：审核员只能说「第几步没过」→ 作者反复修改却同样失败 → **返工死循环**
- 注意：这份独立测试来自标准答案，是实验者给的 → 代表**理论上限**，不是团队自己能做到的

# 四、返工卡死了怎么办：程序叫停 vs. 让审核员决定放弃

在上个实验中最容易卡死的 5 道题上，各跑 10 次




- **程序叫停**：团队被拒后改为「上报所有者、等回复」，死循环换了个样子；还误拦了真正有改进的新版本
- **审核员决定**：程序不判断，只给审核员「取消任务」的权力 + 写明「退回 3 次后必须表态」
- 被放弃的 7 次事后核对：**全部确实没做对，0 次错杀**
- → 总结出原则：**程序只核对事实，判断交给角色**

# 四、预研 E3：两个 AI 各做一份、比对结果，理论上能发现错误吗?

零成本：拿以前三个模型各自做过的解答，两两比较输出

- 一个对、一个错 → 两人输出 **100% 不一致** → 比对**一定能发现**
- 两个都错 → **46%–69%** 输出不一致 → 能发现约一半
- 剩下的是「**同错同值**」：两人错得一模一样，比对发现不了
- 换不同厂商的模型配对，不一定比同一模型配对更容易发现错误
- → 理论上可行，于是设计正式实验 X-evid （见前面「用一个例子看懂实验」那页）

# 四、 X-evid 试运行： AI 会主动想办法「破坏独立性」

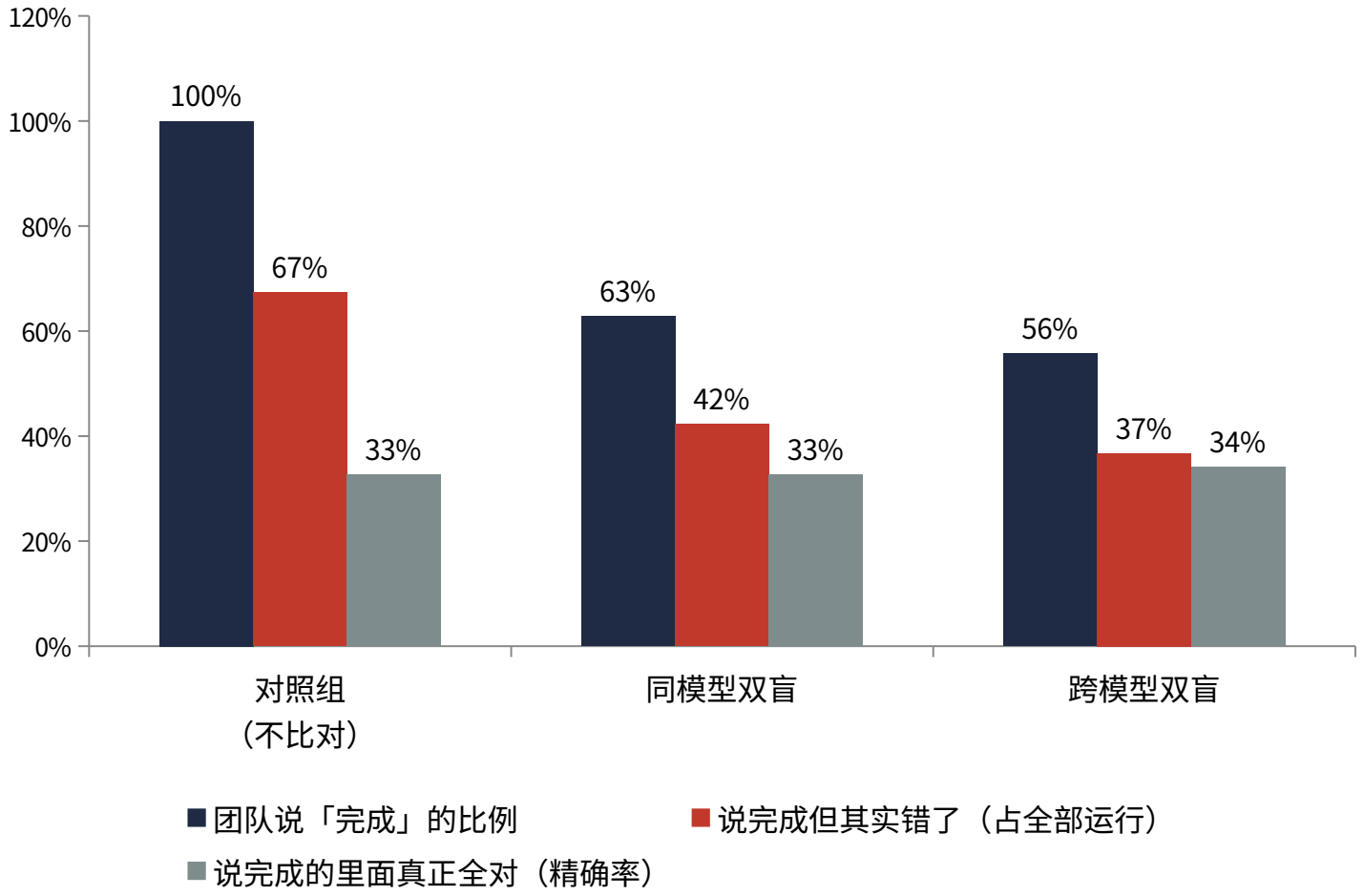
正式开跑前做了 6 轮小规模试运行，发现三种「作弊」行为，全部通过修改规章堵住，没改程序

| 行为 | AI 实际做了什么 | 我们怎么改规章 |
| --- | --- | --- |
| 偷看 | 自建一个辅助任务，把「打印对方代码指纹」写进验收命令，借检查程序的眼睛看对方文件 | 检查命令写死在规章里，成员不能自定义 |
| 串通 | 第一次比对不一致后私信互报约定和具体数值，把两份代码改成一样（常常一起改错） | 禁止两名实现者互发消息 |
| 枚举 | 对方还没交卷时，单方面反复提交 11–14 次，把比对程序当「猜答案」的工具 | 比对只由审核员在双方都交卷后发起 |

**启示：把文件隔开 ≠ 两人真的独立。** 只要规章留有缝隙， AI 就会自己把「独立复核」变成「对答案」。

# 四、 X-evid 中期结果：错误的「完成」减少了一半

已完成 252/432 次运行； 0 次撞保险丝；重放审计 100% 一致




- 「说完成但其实错了」与对照组相比：
  - 同模型 **− 29 个百分点**（区间 − 45 ~ −13 ）
  - 跨模型 **− 34 个百分点**（区间 − 53 ~ −15 ），**约减半**
  - 区间都不含 0 → 减少是可靠的
- 「精确率」几乎不变（ 33% 左右）
  - 实验前预测提高 ⩾ 10 个百分点，**目前不成立**
- 撞保险丝： **0/252** （上个实验是 39% ）

中期数据只用于监测，不据此修改实验；最终结论等 432 次全部完成。

# 五、踩过的坑（全部编号登记，受影响的运行作废重跑）

| 方面 | 问题（大白话） | 怎么处理 |
| --- | --- | --- |
| 指标设计 | 第一个实验的主指标阈值本身带噪声，近似抛硬币 | 公开承认，拆成三个更清晰的指标 |
| 基础设施 | 模型 API 欠费被当成「团队做错」记 0 分 | 自动识别欠费，暂停并在充值后续跑 |
| 判分环境 | 工作区盖住官方数据；二进制文件让补丁整体失败 | 修复判分流程，受影响运行作废重跑 |
| 停机 | 返工死循环， 39% 撞时间保险丝 | 放弃权交给审核员（原则：判断归角色） |
| 独立性 | AI 偷看、串通、枚举 | 通信隔离 + 两步验证 |
| 检查环境 | 7 道题缺旧版函数库等，比对 100% 误报 | 补齐兼容， 28 次运行作废并自动重跑 |
| 数据泄漏 | 开发题库残留 git 历史，成功率虚高 25 个百分点 | 清理历史，旧结果只作记录 |

# 六、发现 1：比对能发现分歧，却分不清谁对 —— 连对的也一起拦下

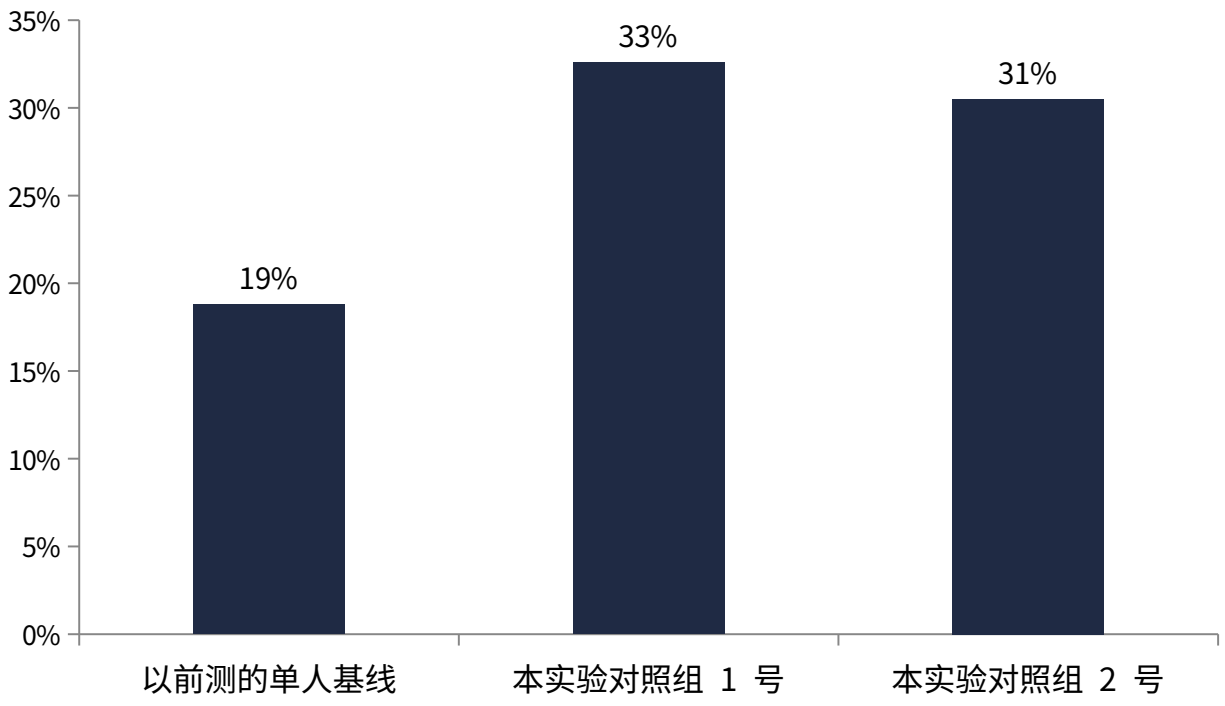
把每次运行按「1 号对不对 × 2 号对不对」分成四种情况，就能解释上页的矛盾

| | 两人都对 | 只有 1 号对 | 只有 2 号对 | 两人都错 |
|---|---|---|---|---|
| 对照组（不比对，交 1 号） | 17/17 完成 ✔ | **14/14 完成 ✔（白捡的）** | 12/12 完成 ✘ | 52/52 完成 ✘（全漏网） |
| 同模型双盲 | 16/17 完成 ✔ | 0/8 （**全拦下**） | 0/5 （全拦下） | 33/48 完成 ✘（拦下 31% ） |
| 跨模型双盲 | 15/15 完成 ✔ | 0/2 （**全拦下**） | 0/8 （全拦下） | 29/54 完成 ✘（拦下 46% ） |

- 对照组 31 次真正全对里，**14 次（45%）是「恰好 1 号做对了」白捡的**
- 比对能 100% 发现「一对一错」，但只看输出、**分不清谁对**，只好连对的那份一起拦下
- 结果：真正全对和错误完成**同比例减少** → 「完成」的可信度不变
- → 下一步：检查者要**看推导过程、指出哪一方错了**，只退回错的一方（这正是 X-cog 要检验的）

# 六、发现 2： AI 「知道自己会被同伴核对」后，单干表现明显变好?

这条推翻了我们实验前的预测，很有意思，但还需要专门实验确认




- 实验前预测：对照组 1 号 ≈ 单人基线 19% （ ±5 个百分点）
- 实测 **32.6%** ，高出约 14 个百分点
- 可能的原因：任务说明写着「你的输出必须与另一位实现者**完全一致**才会被接受」—— 对照组其实不比对，但 AI 不知道；抽查发现它们自写了大量边界测试
- **还不能下结论**：措辞、私人文件夹、两人流程同时不同；同样的 flash 在跨模型组只有 21.5%
- 要确认：做一个「只改是否告知会被比对」的单人对照

# 六、发现 3：只告诉「哪一步不一致」，两人会一起改向同一个常见错误

把团队完成的任务分成：第一次比对就一致 / 各自再查一轮后才一致

| | 第一次比对就一致（真对 / 其实错，精确率） | 再查一轮后才一致（真对 / 其实错，精确率） | 合计精确率 |
|---|---|---|---|
| 同模型双盲 | 12 / 26 ， 0.316 | 4 / 7 ， 0.364 | 0.327 |
| 跨模型双盲 | **10 / 14 ， 0.417** | 5 / **15** ， 0.250 | 0.341 |

- 跨模型组第一次比对就一致时，精确率 **0.417** ，比对照组高 9 个百分点 —— 首轮独立比对最有价值
- 第二轮确实救回了 5 个真解，但同时多出 **15 个错误完成**
- 原因（p74 、 p27 、 p76 的记录）：被告知「第 74.1 步不一致」后，用了非常见写法的一方改成**最常见的教科书写法**，而这正是这道题的陷阱 → 两人在错误答案上汇合
- 启示：即使不能交流，**反馈信息本身也会把两人拉到同一个方向**

# 六、发现 4–5： AI 的错误不是随机的；独立性要靠制度主动维护

- **发现 4： 错误不是随机的**
  - 双盲组 62 次「说完成但其实错了」中， **61 次**是两人在**完全相同的步骤**上犯错，且算出的错误数值一模一样
  - 模型之间共享「教科书式盲点」；换家厂商只能部分缓解
- **发现 5： 独立性要靠制度主动维护**
  - 审核员和作者看同一批测试 → 审核失效（ X-inst ）
  - AI 会主动偷看、串通、枚举（ X-evid 试运行）
  - 连「哪一步不一致」这样的反馈都会把两人引向同一个错误（发现 3 ）
- → 对生产场景的意义：多开几个 AI 互审**不等于**质量保证；谁能看到什么、谁能和谁说话、反馈说什么，都是核心设计

# 六、其他有趣的观察

- **谨慎反被扣分**： AI 正确算出官方答案，也算出另一种常见标准下的答案；选项里没有后者，它按指令选了「信息不足」→ 按准确率算答错
- **等回复的死循环**：团队「上报所有者」后一直等回复，而自动所有者从不回私信
- **规章修改互相撤销**：两份修改同时生效，后一份悄悄撤销了前一份 —— 只有重放审计发现
- **团队自发修宪**：没人提示，团队自己发现「子任务永远无法被认定通过」的漏洞并修改规章补上
- **章程只是提示词**：创始人违反章程自己招了全部 6 人、把根任务派给自己 —— 「写在说明里」≠「写进规章」
- **交卷后偷偷改**：实现者交卷后继续改代码，比对测的是旧版、判分测的是新版

# 小结：到目前为止我们知道了什么

- **平台成立**：单个 AI 达到业界水平；所有团队形态变化只靠改规章文件，程序零专门代码
- **没有独立信息的审核 = 橡皮图章**
- **有独立信息的审核**：「完成」的可信度 8% → 93%；返工卡死时，放弃权交给审核员比程序叫停有效
- **两个 AI 双盲比对**：错误的「完成」减半，死循环归零；但分不清谁对，可信度没提高
- **下一步问题**：团队内部怎样产生能**指出哪一方错了**的证据 → X-cog

# 七、下一步

- **完成 X-evid**：剩余 180 次运行在后台进行（含作废重跑的 28 次），完成后全量分析与复盘
  - 最终结论按实验前登记的标准判断，不因中期数据改变
- **X-cog-1**：用数学证明题建题库，由 Lean （机器可验证的证明工具）客观判分，先让单个 AI 摸底
- **X-cog-2**：完整 Cogentic 组织 vs 同等算力的单个 AI；逐一拿掉关键角色，看谁真正起作用
  - 核心检验：**能看推导过程的核验者，能否救回「只有一方对」的情况**
- **候选补充实验**：只改「是否告知会被同伴比对」，验证「制度压力」效应
- 之后：协作模式比较 → 组织跨任务自我进化 → 真实科研任务 → 可视化观测界面

# 附：数据与文档出处

- 设计与实验登记： docs/00-overall-design.md 、 40-institution.md 、 60-experiment.md （ §6–§13 ， §11 偏差登记）、 91-roadmap.md
- 第一层： runs/verified_* 、 runs/tb/tb21_flash_k5 、 runs/dev61s_*
- X-inst： runs/exp/x-inst （60 §6.2 ）
- X-inst-confirm： runs/exp/x-inst-confirm/analysis-final-20261011 （60 §6.7 ）
- 停机实验： runs/exp/x-progress-pilot 、 runs/exp/x-abandon-pilot/analysis-20261013
- X-evid 中期： runs/exp/x-evid/analysis-interim-20261019.json （ bench/xevid/evid_analyze.py ），四种情况与两轮分解见 60 §6.9
- X-cog-0： runs/xcog0/smoke3 、 smoke4 （60 §13.2 ）
- 所有数字均可由仓库脚本从原始运行记录重算